KOSTA AI LAB · AI NATIVE SDLC

바이브코딩

기획 단계에서 배포까지 — 비개발자를 위한 1일 과정

코드를 직접 작성하지 않고, 자연어만으로 소프트웨어를 구현합니다

기획 설계 개발 배포

과정 프로젝트 — 내 앱 직접 구현

기획부터 배포 URL 확보까지 자연어만으로 완성하는 전 과정을 경험합니다

💬

1단계 — 자연어 첫 시도

'할일 관리 앱을 만들어 주십시오'와 같은 한 문장만 요청해 앱의 기본 형태를 구성합니다

✨

2단계 — 기획 기반 재구성

요구사항, UX 플로우, 디자인 시스템을 적용하여 결과물 품질을 향상시킵니다

🔧

3단계 — 데이터 영속화

Supabase와 연동해 입력 데이터를 영구 저장합니다

🌐

4단계 — 라이브 배포

Vercel에 배포해 외부 사용자와 공유할 URL을 확보합니다

기획자·PM 직무에서 바이브코딩이 제공하는 가치

개발 업무를 대체하지 않고, 기획 의도를 동작하는 실물로 전달합니다

⏱️

IT 요청 대기 시간 단축

외주나 개발팀에 맡겨 수일 걸리던 프로토타입을 당일에 직접 구현하고 검증할 수 있습니다.

🎯

기획 의도의 실물 전달

문서나 와이어프레임 대신 동작하는 프로토타입으로 전달해 요구사항 해석의 오차를 줄입니다.

🤝

개발 조직과의 공통 언어 확보

CLAUDE.md, 요구사항 정의서, 디자인 토큰 등 표준 산출물로 개발팀과 더 원활하게 소통할 수 있습니다.

🏆

도메인 지식의 가치 상승

AI가 코드를 생성할수록 고객과 비즈니스를 이해하고 정의하는 역량이 중요해집니다.

오늘의 시간표

7개 블록으로 바이브코딩 전 과정을 체험합니다 (09:30-17:30, 점심 12:30-13:30)

시간	단계	내용
09:30 - 10:30	입문	오프닝 / 바이브코딩 개념 / 환경 점검
10:30 - 11:30	도구	Claude Code 설치 / Supabase·Vercel plugin 설치
11:30 - 12:30	체험	첫 바이빙 — 한 문장으로 만든 첫 앱
12:30 - 13:30	점심	
13:30 - 14:30	기획	요구사항·UX 설계 / 개인 기획안 정리
14:30 - 15:30	설계	프롬프트·Plan Mode / 디자인 시스템·CLAUDE.md
15:30 - 16:30	구현	기획서 기반 앱 구현 (Plan Mode + CLAUDE.md)
16:30 - 17:30	통합	Supabase plugin DB / Vercel plugin 배포 / 동료 시연 / 클로징

학습 목표

과정 수료 후 수행 역량

- 바이브코딩의 개념과 비개발자에게 가능한 영역을 설명할 수 있습니다
- Claude Code를 설치하고 자연어만으로 웹앱을 생성할 수 있습니다
- 요구사항과 UX를 체계적으로 정리하여 결과물 품질을 높일 수 있습니다
- 디자인 시스템과 CLAUDE.md로 전문적인 UI와 프로젝트 맥락을 구축할 수 있습니다
- Plan Mode와 Skills로 효율적으로 반복 개선할 수 있습니다
- Supabase로 데이터 저장 기능을 연동할 수 있습니다
- 완성된 앱을 Vercel에 배포하여 실제 URL을 얻을 수 있습니다

OVERVIEW

과정에서 다루지 않는 내용

기대치를 명확히 하여 실무 적용 시 혼선을 줄입니다

🏗️

엔터프라이즈 아키텍처

대규모 트래픽 처리, 마이크로서비스, 분산 시스템 설계는 본 과정 범위를 벗어납니다.

🔒

상세 보안 감사

코드 보안 감사, 침투 테스트, 취약점 분석은 보안 전문가·도구의 영역입니다. 본 과정은 기본 주의사항 수준만 다룹니다.

📋

법무·컴플라이언스 승인

사내 AI 사용 정책, 데이터 보호 규제, 라이선스 검토는 법무·컴플라이언스 조직과 별도 협의가 필요합니다.

💼

운영 단계 (5단계)

모니터링, 로깅, CI/CD, SRE는 오늘 다루지 않습니다. 프로토타입에서 운영 배포로 넘어갈 때 별도 학습이 필요합니다.

🧪

품질 보증 체계

AI가 생성한 코드의 단위 테스트·통합 테스트·QA 자동화는 후속 과정의 주제입니다.

🚨

장애 대응·복구

프로덕션 장애 대응, 백업·복구 전략, SLA 관리는 운영팀·DevOps 영역입니다.

강사 소개: 윤성열

(주)프렌티스 대표이사 · 공학박사

17년 이상	8년 이상	특허 4건	공학박사
SW 아키텍처	기업 교육/컨설팅	AI/에이전트 시스템	가천대 전자계산학

주요 경력

2025~현재	대표이사	(주)프렌티스 — AI 플랫폼 기술개발 총괄, Spectra/CodeRay 제품화
2023~현재	실장	(주)닥터소프트 — SaaS Management Platform 개발 총괄
2018~현재	전문강사	KOSTA — AI/클라우드/SW개발 기업 교육, 과정기획 자문
2020~2022	연구소장	(주)네비웍스 — 메타버스 서비스 개발 총괄
2017~2018	겸임교수	가천대학교 — 대학원 융합서비스/모바일프로그래밍

교육/컨설팅 고객

KT / KTDS	AI 기반 SW 개발, Agentic AI
SK플래닛	AI 기반 SW 개발 방법론
글로벌널리지(AIVLE)	AI, 컨테이너, Agentic AI
KLCNS / 페타시스	Private AI 도입 컨설팅
경남대 / 가천대	디지털 기획, SW 교육

실습 환경 안내

다음 도구가 준비되었는지 확인하십시오

Terminal bash

```
# 1) Claude Code 설치 (네이티브, Node.js 불필요)
curl -fsSL https://claude.ai/install.sh | bash

# 2) 실행하면 브라우저가 열립니다 — 본인 Claude 계정으로 로그인
claude

# 3) 설치와 인증 확인
claude --version
# 세션 안에서 /status 로 인증 상태를 봅니다

# 구독 대신 API 키를 쓴다면
# export ANTHROPIC_API_KEY=sk-ant-...
```

🔑

Claude 계정

Pro·Max 구독이면 claude 실행 후 브라우저 로그인으로 끝납니다. 구독이 없으면 Console에서

🔧

Git + GitHub 계정

git --version 확인. GitHub 계정은 배포 연동에 사용됩니다. user.name / user.email 설정이

▲

Vercel 계정

vercel.com에서 무료 가입합니다. GitHub 연동을 권장합니다. 실제 URL 배포에 사용됩니다

🗄️

Supabase 계정

supabase.com에서 무료 가입합니다. 오후 데이터베이스 연동 실습에 사용됩니다

01

바이브코딩 개념

- ✓ 바이브코딩의 정의와 탄생 배경을 설명할 수 있습니다
- ✓ 기존 코딩 방식과 바이브코딩의 차이를 비교할 수 있습니다
- ✓ AI 코딩 도구 생태계에서 Claude Code의 위치를 이해합니다

⏱ 25분

LO1 · 바이브코딩 개념

“There's a new kind of coding I call ‘vibe coding’, where you fully give in to the vibes, embrace exponentials, and forget that the code even exists.”

코드의 존재 자체를 잊고, 자연어로 의도만 전달하여 소프트웨어를 만드는 새로운 개발 방식입니다. AI에게 전적으로 맡기고, 결과물의 느낌(vibe)으로 방향을 잡아 나갑니다.

Andrej Karpathy, 2025년 2월

AI 코딩 시장의 현주소

개발자 대다수가 AI 도구를 사용하고 있지만, 품질 리스크도 함께 증가하고 있습니다

84%

개발자 AI 코딩 도구 사용률

Stack Overflow 2025

55%

코딩 속도 향상

GitHub 2025

45%

보안 취약점 포함 비율

Veracode 2025

기회와 위험이 공존합니다. 제대로 배워야 하는 이유입니다.

Stack Overflow Developer Survey 2025; GitHub Copilot Research 2025; Veracode 2025

용어 창시자의 역설

Karpathy가 nanochat 프로젝트에서 수동 코딩으로 회귀하였습니다

- nanochat: 순수 수작업 코드 (hand-written, zero vibe)
- 에이전틱 코딩 시도: "전체적으로 도움이 되지 않았다(net unhelpful)"
- 바이브코딩은 "주말 프로젝트용(throwaway weekend projects)"

바이브코딩의 한계를 인식하고, 적절한 영역에서 활용하는 것이 핵심입니다.

Andrej Karpathy (@karpathy), X(Twitter) Post, Feb 2025; Gary Marcus

기존 코딩 vs 바이브코딩

개발 방식의 패러다임이 근본적으로 바뀌고 있습니다

항목	기존 코딩	바이브코딩
코드 작성	개발자가 직접 한 줄씩 작성합니다	자연어로 의도를 전달하면 AI가 생성합니다
디버깅	에러 메시지를 읽고 코드를 수정합니다	오류 상황을 설명하면 AI가 원인을 찾아 수정합니다
설계	UML, 설계 문서를 직접 작성합니다	요구사항을 대화로 전달하면 AI가 구조를 제안합니다
학습 곡선	언어 문법, 프레임워크 학습이 필수입니다	문제 해결 능력과 프롬프트 작성이 핵심입니다
개발 속도	경험이 쌓일수록 개발 속도가 빨라집니다	프롬프트 품질에 따라 초보자도 빠르게 개발합니다

회색지대 — AI가 채워주는 빈 공간

"내가 정하지 않은 부분은 AI가 채웁니다 — 이 빈 공간을 잘 채우게 하는 사람이 AI를 잘 활용합니다"



내가 정한 영역

기능 목록, 핵심 사용자, 톤앤매너 — 명확히 지시한 부분은 AI가 그대로 반영합니다.



회색지대

버튼 색상, 에러 메시지 문구, 레이아웃 간격 — 내가 미처 정하지 못한 빈 공간을 AI가 추측으로 채웁니다.



AI가 채운 결과

빈 공간을 잘 채우면 작품이 되고, 못 채우면 어색한 결과물이 됩니다. 구체적일수록 회색지대가 줄어듭니다.

AI 코딩 도구 생태계

도구의 형태에 따라 세 가지로 나뉩니다



IDE 기반

Cursor, Windsurf, GitHub Copilot 등 에디터에 내장되어 코드를 자동 완성하고 수정합니다. 기존 개발 환경과 자연스럽게 통합됩니다.



터미널 기반

Claude Code, Codex CLI 등 터미널에서 동작합니다. 파일 시스템 전체를 직접 읽고 수정하며, 에이전틱 방식으로 작업을 수행합니다.



노코드 플랫폼

Lovable, Replit Agent, Bolt 등 브라우저에서 대화만으로 앱을 생성합니다. 코드를 전혀 보지 않고도 결과물을 얻을 수 있습니다.

Claude Code의 4가지 핵심 특성

터미널에서 동작하는 에이전틱 코딩 도구입니다

- 터미널 네이티브: IDE 없이 터미널에서 직접 실행되며, 어떤 에디터와도 독립적으로 사용할 수 있습니다
- 에이전틱(자율) 동작: 한 번 지시하면 AI가 스스로 판단해 파일을 읽고 수정·생성하며 명령어까지 실행합니다
- 프로젝트 메모리: CLAUDE.md 파일에 프로젝트 규칙과 맥락을 저장해 일관된 결과를 보장합니다
- 확장성: 플러그인, 커스텀 명령어, 서브에이전트(독립 작업자)로 기능을 확장할 수 있습니다

바이브코딩의 적합 영역과 부적합 영역

바이브코딩이 효과적인 경우와 주의가 필요한 경우를 구분합니다

적합한 영역

- ✔ 프로토타입 및 MVP 빠른 제작
- ✔ CRUD 기반 웹 애플리케이션
- ✔ 자동화 스크립트 작성
- ✔ 데이터 분석 및 시각화
- ✔ 학습 목적의 코드 탐색

부적합한 영역

- ✖ 실시간 거래 시스템(금융, 증권)
- ✖ 항공/의료 등 안전 필수 소프트웨어
- ✖ 대규모 레거시 시스템 마이그레이션
- ✖ 고도로 최적화된 저수준 프로그래밍
- ✖ 법적 감사가 요구되는 코드베이스

체크포인트

다음 섹션으로 넘어가기 전에 확인합니다

- ✔ 바이브코딩의 정의를 한 문장으로 설명할 수 있습니까?
- ✔ 기존 코딩과 바이브코딩의 핵심 차이 3가지를 말할 수 있습니까?
- ✔ AI 코딩 도구의 세 가지 분류(IDE, 터미널, 노코드)를 구분할 수 있습니까?

02

Claude Code 설치와 기본 사용

- ✓ Claude Code를 설치하고 인증을 완료할 수 있습니다
- ✓ 기본 도구 6가지의 역할을 이해합니다
- ✓ 권한 시스템을 설정하여 안전하게 사용할 수 있습니다

⏱ 25분

설치 방법

네이티브 설치를 권장하며, Node.js 없이도 설치할 수 있습니다

macOS / Linux — bash

```bash
# 네이티브 설치 (권장, Node.js 불필요)
curl -fsSL https://claude.ai/install.sh | bash

# 설치 확인
claude --version
```

대안 1: npm — bash

```bash
# npm (Node.js 18+ 필요)
npm install -g @anthropic-ai/claude-code
```

대안 2: Homebrew — bash

```bash
# Homebrew (macOS)
brew install claude-code
```

네이티브 설치는 **~/.local/bin/claude**에 설치되며, Node.js 의존성이 없어 가볍습니다. Windows는 WSL 환경에서 동일하게 설치합니다.

인증 방식 3종

OAuth, API 키, AWS Bedrock 중 상황에 맞는 방식을 선택합니다

번호	인증 방식	설정 방법	적합한 경우	비용
1	OAuth (개인 구독)	claude 실행 시 브라우저 인증	개인 학습, 장기 사용	Pro/Max 구독 포함
2	API 키 (종량제)	Console 로그인 → API Key 발급 → ANTHROPIC_API_KEY 설정	단기 체험, 자동화·CI/CD	사용량 기반
3	AWS Bedrock	AWS 자격증명 설정	기업 환경, 보안 요구사항	AWS 요금

💡 **실습은 1번(개인 구독 로그인)을 권장합니다.** claude를 실행하면 브라우저 인증 한 번으로 끝나고, 사용량은 구독 한도 안에서 관리됩니다.
구독이 없으면 2번을 씁니다. Anthropic Console에서 발급한 키를 **ANTHROPIC_API_KEY** 환경변수에 넣고, 키는 저장소에 커밋하지 않습니다. 사용량은 세션 안에서 **/cost**로 확인합니다.

기본 도구

Claude Code는 요청을 분석하여 적절한 도구를 자동으로 선택합니다

📖 **Read**	파일 내용을 읽습니다
✏️ **Write**	새 파일을 생성합니다
🔧 **Edit**	기존 파일을 수정합니다
🔨 **Bash**	터미널 명령을 실행합니다
🔍 **Glob / Grep**	파일과 텍스트를 검색합니다

슬래시 커맨드

대화 중 /를 입력하면 사용 가능한 명령어가 표시됩니다

커맨드	기능	사용 시점
/help	사용 가능한 명령어 목록을 표시합니다	기능이 기억나지 않을 때
/clear	대화 히스토리를 초기화합니다	새로운 작업을 시작할 때
/compact	대화 내용을 요약하여 컨텍스트를 확보합니다	대화가 길어져 응답이 느려질 때
/cost	토큰 사용량과 비용을 확인합니다	비용을 추적할 때
/model	AI 모델을 변경합니다	다른 모델을 시도할 때
/init	CLAUDE.md를 자동 생성합니다	프로젝트 설정을 시작할 때
/plugin	플러그인을 관리·설치합니다	외부 도구(Vercel, Supabase 등) 연동 시
claude doctor	설치 상태를 진단합니다 (CLI 명령)	문제가 의심될 때 터미널에서 실행

/를 입력하면 전체 커맨드 목록이 나타나며, 타이핑으로 필터링할 수 있습니다. 오늘 자주 사용할 커맨드: **/clear**, **/compact**, **/init**

3단계 권한 시스템

안전한 사용을 위해 도구별 실행 권한을 세밀하게 제어합니다

- 자동 허용(allow): 기본 모드에서 Read, Glob, Grep 등 읽기 전용 도구는 확인 없이 실행됩니다
- 확인 필요(ask): Write, Edit, Bash 등 변경 도구는 실행 전 사용자 승인을 요청합니다
- 거부(deny): 위험한 명령어(rm -rf, 외부 스크립트 실행 등)는 기본적으로 차단됩니다
- permission mode(Shift+Tab으로 순환)에 따라 allow/ask가 달라집니다 — auto-accept는 ask를 자동 승인합니다

.claude/settings.json — json

```
// .claude/settings.json  (기본 permission mode 기준)
{
  "permissions": {
    "allow": ["Bash(npm test)", "Bash(npm run lint)"],
    "ask":   ["Write", "Edit"],
    "deny":  ["Bash(rm -rf *)", "Bash(curl * | bash)"]
  }
}
```

사내 데이터 경계와 보안 원칙

실무 적용 전에 반드시 검토할 데이터 경계·정책 항목입니다

입력 금지 데이터

사내 소스 코드, 고객 PII, 영업 비밀, 내부 아키텍처 문서 등은 교육 과정에서 사용하지 않습니다.

실무 적용 전 확인

사내 프록시·AWS Bedrock 경유 여부, 로그 보관 정책, Anthropic 학습 opt-out 설정을 IT·보안팀과 먼저 확인합니다.

컴플라이언스

사내 AI 사용 가이드라인이 있으면 준수합니다. 없으면 수료 후 보안팀에 가이드라인 수립을 제안합니다.

- 실습은 공개 예제(할일 앱·기획안 등)로만 진행합니다 — 사내 정보 입력 금지
- API 키는 환경변수로만 두고 코드·저장소에 남기지 않습니다

체크포인트

다음 섹션으로 넘어가기 전에 확인합니다

- ✔ Claude Code가 정상적으로 설치되어 claude --version이 실행됩니까?
- ✔ OAuth 인증을 완료하고 Claude Code에 질문을 보낼 수 있습니까?
- ✔ Read, Write, Bash, Glob, Grep 도구의 역할을 각각 설명할 수 있습니까?
- ✔ 권한 설정 파일(.claude/settings.json)의 구조를 이해했습니까?

00

실습 1: 그냥 바이브

- 자연어 프롬프트로 웹앱을 생성합니다
- Claude가 선택한 기술 스택을 관찰합니다
- 결과의 한계를 직접 확인합니다

⏱ 25분

BLOCK 1 자연어로 앱 생성하기

실습 절차

아래 프롬프트를 Claude Code에 입력하여 할일 관리 앱을 생성합니다. 스택을 지정하지 마십시오. Claude가 알아서 선택하도록 합니다.

```
할일 앱 만들어줘
```

관찰 포인트

Claude가 어떤 기술 스택을 선택하는지, 어떤 순서로 파일을 생성하는지 관찰합니다.

코드 작성 없이 동작하는 앱이 만들어집니다

결과 관찰

Claude가 생성한 앱을 직접 실행하고 확인합니다

🔍

기술 스택 확인

Claude가 선택한 프레임워크, 라이브러리, 프로젝트 구조를 확인합니다

🖥️

화면 확인

브라우저에서 앱을 실행하고 UI와 기능을 확인합니다

📁

코드 구조 확인

생성된 파일 목록과 코드를 훑어봅니다

BLOCK 2 기능 추가해보기

실습 절차

기본 앱에 간단한 기능을 자연어로 추가합니다. 한 번에 하나씩 요청합니다.

"삭제 버튼을 추가해줘"

관찰 포인트

한 번에 하나의 기능만 요청하고 결과를 확인합니다. 추가 기능도 시도합니다.

기능이 추가되지만, 디자인은 여전히 기본적입니다

문제 인식

그냥 바이브의 한계를 확인합니다

- ! 디자인이 기본적이고 전문성이 부족합니다
- ! 기능 요구사항이 불명확하여 원하는 결과가 나오지 않을 수 있습니다
- ! 색상, 간격, 타이포그래피에 일관성이 없습니다
- ! 새로고침하면 데이터가 사라집니다
- ! 다음 시간에 이 문제들을 해결합니다

실습 1 체크포인트

다음 섹션으로 넘어가기 전에 확인합니다

- ✔ 앱이 브라우저(http://localhost:3000 등)에서 정상 실행됩니다
- ✔ 할일을 추가하고 삭제할 수 있습니다
- ✔ 결과물의 한계를 1가지 이상 확인했습니다
- ✔ Git 커밋: 'git init과 첫 커밋을 해줘'라고 요청합니다 (롤백 지점 생성)

03

요구사항과 UX 설계

- ✓ 요구사항을 체계적으로 수집하는 방법을 익힙니다
- ✓ 사용자 스토리와 페르소나를 작성합니다
- ✓ UX 플로우와 화면 구성을 설계합니다

⏱ 25분

'그냥 바이브'의 한계

실습 1에서 만든 앱의 한계를 분석합니다

항목	그냥 바이브	요구사항+UX 적용
디자인	기본 HTML 스타일	디자인 시스템 기반 전문 UI
기능	AI가 임의로 결정	사용자 스토리 기반 명확한 기능
구조	단일 파일 또는 임의 구조	컴포넌트 기반 체계적 구조
사용성	고려하지 않음	사용자 페르소나 기반 UX 설계

마크다운(.md) 소개

요구사항·디자인·규칙을 적을 때 쓰는 가벼운 문서 형식입니다

마크다운 기본 문법 markdown

```markdown
# 가장 큰 제목
## 중간 제목
- 목록 항목   **굵게**   *기울임*   `인라인 코드`
```

📝

AI가 구조를 이해

제목과 - 목록만으로 문서 구조가 명확해집니다.

👀

어디서나 읽힘

메모장·VS Code·GitHub에서 서식 그대로 표시됩니다.

📦

Git과 궁합

텍스트 파일이라 변경 이력을 쉽게 추적할 수 있습니다.

오늘 프로젝트에 생길 파일

어떤 파일이 왜 있는지만 알면 됩니다 — 코드 내용은 지금 안 봐도 됩니다

my-app/ text

```
my-app/
├── CLAUDE.md            ← 프로젝트 규칙 (AI 맥락)
├── requirements.md      ← 요구사항 (이번 시간에 만들 것)
├── design-guide.md      ← 디자인 가이드
├── .env.local           ← 비밀값 (숨김, Git 제외)
├── package.json         ← 프로젝트 정보
└── src/                 ← 실제 코드 (Claude가 관리)
```

🖐

직접 열어볼 3개

CLAUDE.md · requirements.md · design-guide.md만 직접 확인·수정합니다.

🤖

Claude가 관리

src/ 안의 코드와 package.json은 대화로 요청만 하면 됩니다.

🔒

건드리면 안 됨

.env.local에 API 키가 들어갑니다. 노출·커밋 금지입니다.

MVP 제외 리스트 — Before / After

기능을 늘리지 말고 빼야 합니다. 제외 리스트가 있어야 AI와 본인이 스코프를 지킵니다.

Before — 풀어 놓은 요구사항	After — MVP 3개 + 제외 리스트
할일 추가, 완료, 삭제, 마감일, 우선순위, 라벨, 댓글, 통계, 알림, 다국어, 다크모드, 협업...	MVP: 추가 / 완료 / 삭제 (3개만)
구현 범위가 모호합니다. AI가 임의로 절반만 만들거나 부풀립니다.	제외: 마감일·라벨·댓글·통계·알림·다국어·다크모드 (확장 과제로 분리)
결과: 첫날 안에 못 끝내 동작을 확인할 시간이 부족합니다.	결과: 동작·배포·시연까지 1일에 가능. 욕심은 다음 회차에.

대화로 요구사항 만들기

직접 문서를 작성하지 않습니다. Claude에게 질문을 받으며 자동으로 정리합니다

할일 관리 앱을 만들 건데, 요구사항을 같이 정리해줘. 내가 대답하면 문서로 만들어줘.

AI

좋습니다. 몇 가지 질문을 드리겠습니다.

1. 이 앱을 주로 누가 사용합니까?
2. 가장 중요한 기능 3가지는 무엇입니까?
3. 모바일에서도 사용해야 합니까?
4. 데이터를 서버에 저장해야 합니까?

직장인이 씁니다. 빠른 입력과 우선순위가 중요합니다. 모바일은 필수이고 저장은 나중에 합니다.

AI

requirements.md를 생성하겠습니다.

대상: 직장인 (20-40대)
핵심 기능: 빠른 할일 추가, 우선순위 관리, 완료 처리
제약: 모바일 반응형 필수
후순위: 서버 저장 (추후 추가)

"질문을 많이 해줘" — 비개발자 검증 패턴

백지에 요구사항을 쓰는 대신 질문에 답합니다 — 숨은 요구까지 함께 찾아냅니다

claude (REPL) bash

```
# 인터뷰 모드를 켜는 한 줄
> 할일 관리 앱을 만들건데 요구사항 같이 정리해줘.
> 문서로 남겨주고, 질문을 많이 해줘.

# 결과: AI가 10~15개 질문을 던지며 인터뷰 진행
# 답변을 누적하면 requirements.md가 자동 채워짐
```

표면 요구사항	숨은 요구(Hidden Wish)
1초 안에 등록하고 싶습니다	잊기 전에 적어야 합니다
글자가 좀 더 컸으면	출퇴근길 흔들리는 차에서 봅니다
버튼이 눈에 잘 띄게	급할 때 한 손으로 사용합니다

비즈니스 분석

앱의 목적, 대상 사용자, 핵심 가치를 정의합니다

🎯

목적 정의

누가, 왜, 어떤 문제를 해결하는지 명확히 합니다. 할일 앱: 직장인의 업무 관리 효율화

👤

대상 사용자

사용자의 특성, 니즈, 기술 수준을 파악합니다. 직장인, 30대, IT 비전공자

💎

핵심 가치

경쟁 제품과 차별화되는 핵심 가치를 정합니다. 3초 내 할일 추가, 한눈에 현황 파악

사용자 스토리 자동 생성

요구사항을 전달하면 Claude가 사용자 스토리 형식으로 변환합니다

방금 정리한 요구사항을 사용자 스토리 형식으로 바꿔 주세요. 각 스토리에 수락 조건도 넣어 주세요.

AI

user-stories.md를 생성하겠습니다.

1. 사용자로서, 할일을 3초 내에 추가할 수 있습니다
   수락 조건: Enter 키로 즉시 추가, 입력 후 포커스 유지

2. 사용자로서, 우선순위별로 할일을 볼 수 있습니다
   수락 조건: 높음/보통/낮음 필터, 색상으로 구분

3. 사용자로서, 완료된 할일을 체크할 수 있습니다
   수락 조건: 클릭 한 번으로 토글, 취소선 표시

사용자 페르소나

구체적인 사용자 인물을 설정하여 의사결정의 기준으로 삼습니다

👩‍💻

김지현 (32세, 마케팅 팀장)

매일 10개 이상의 업무를 관리합니다. 회의 중에도 빠르게 할일을 추가하고 싶습니다. 디자인 감각이 있어 깔끔한 UI를 선호합니다.

👨‍💼

이준호 (28세, 주니어 개발자)

사이드 프로젝트와 회사 업무를 분리하여 관리합니다. 키보드 중심 조작을 선호합니다. 다크 모드를 필수로 원합니다.

UX 플로우

사용자가 앱을 사용하는 흐름을 설계합니다

- 앱 진입: 오늘의 할일 목록이 바로 표시됩니다
- 할일 추가: 상단 입력 필드에 텍스트를 입력하고 Enter를 누릅니다
- 우선순위 설정: 할일 옆 드롭다운에서 높음/보통/낮음을 선택합니다
- 완료 처리: 체크박스를 클릭하면 취소선과 함께 하단으로 이동합니다
- 삭제: 스와이프 또는 삭제 버튼으로 항목을 제거합니다

화면 구성 자동 설계

요구사항과 사용자 스토리를 기반으로 Claude가 화면을 설계합니다

이 요구사항과 사용자 스토리를 바탕으로 화면 구성을 설계해 주세요. 메인 화면 하나에 모든 기능을 넣어 주세요.

AI

화면 구성을 설계하겠습니다.

[헤더] 앱 제목 + 필터 (전체/진행중/완료)
[입력 영역] 텍스트 입력 + 우선순위 드롭다운 + 추가 버튼
[할일 목록] 카드형 리스트
  - 체크박스 / 제목 / 우선순위 배지 / 마감일 / 삭제
[하단 요약] 전체 N개, 완료 N개, 남은 N개

이 구성으로 컴포넌트를 분리하여 구현하시겠습니까?

좋아, 그대로 구현해줘.

"안 3개 / 1A 2B 3A" — 비개발자 의사결정 패턴

무엇이 좋은지 모르니 결정이 안 됩니다. AI에게 안을 늘어놓게 하고, 답변은 한 줄(1A 2B 3A)로 끝냅니다

```
claude (REPL)                                                    bash

> 화면 레이아웃 안을 영역별로 3개씩 줘.
> 헤더, 본문, 액션 버튼 영역에 대해
> 내가 1A 2B 3A 형태로 답할 수 있게 해줘.
```

①

헤더 3안

A. 미니멀 / B. 컬러 강조 / C. 검색바 우선

②

본문 3안

A. 카드 그리드 / B. 리스트 / C. 캔반 보드

③

액션 3안

A. FAB 우하단 / B. 헤더 버튼 / C. 인라인 입력

답변: "1A 2C 3A" — 끝

기술 원리 잠깐!

요구사항 문서가 AI 코딩에 미치는 영향을 이해합니다

- 요구사항이 명확할수록 AI가 생성하는 코드의 품질이 높아집니다
- 사용자 스토리는 AI에게 '무엇을 만들어야 하는지' 정확히 전달합니다
- 화면 구성은 AI가 컴포넌트를 분리하는 기준이 됩니다
- UX 플로우는 AI가 상태 관리와 라우팅을 설계하는 근거입니다

체크포인트

다음 섹션으로 넘어가기 전에 확인합니다

- ✔ 요구사항을 Claude에게 요청하는 프롬프트를 작성할 수 있습니까?
- ✔ 사용자 스토리를 '~로서, ~할 수 있습니다' 형식으로 작성할 수 있습니까?
- ✔ 화면 구성을 자연어로 설명할 수 있습니까?

05

프롬프트 전략과 Skills

- ✓ 효과적인 프롬프트 작성법을 익힙니다
- ✓ Plan Mode와 Extended Thinking을 활용합니다
- ✓ 나만의 스킬을 만들고 컨텍스트를 관리합니다

⏱ 25분

좋은 프롬프트 vs 나쁜 프롬프트

구체적인 맥락과 요구사항을 포함하면 결과가 크게 달라집니다

상황	나쁜 프롬프트	좋은 프롬프트
기능 추가	로그인 추가해줘	이메일/비밀번호 기반 로그인을 추가합니다. JWT 토큰으로 인증하고, 로그인 실패 시 에러를 표시합니다
버그 수정	할일이 안 지워져	삭제 버튼 클릭 시 화면에서는 사라지나 새로고침하면 다시 나타납니다. API 호출이 누락된 것 같습니다
디자인	예쁘게 만들어줘	design-guide.md의 디자인 토큰을 적용하여 카드 스타일의 할일 목록으로 변경합니다

Plan Mode 활용

코드를 작성하기 전에 계획을 세우고 검토합니다

- Shift+Tab을 두 번 눌러 권한 모드를 Normal → Auto-Accept → Plan 순서로 전환합니다
- 구현할 작업을 자연어로 입력합니다
- Claude가 단계별 실행 계획을 먼저 제시합니다 (코드 변경 없이)
- 계획을 검토하고 수정이 필요하면 피드백을 제공합니다
- 계획이 확정되면 승인하여 실제 구현을 시작합니다

Extended Thinking

복잡한 문제에 대해 Claude가 더 깊이 생각하도록 유도합니다

키워드	사고 깊이	사용 시점
think	기본 사고	일반적인 코드 작성, 간단한 수정
think hard	깊은 사고	복잡한 로직, 다중 파일 수정
ultrathink	최대 사고	아키텍처 설계, 대규모 리팩토링

- → 프롬프트에 think / think hard / ultrathink 키워드를 넣으면 더 깊이 생각합니다
- → 별도 토글 단축키는 없습니다. 원하는 순간에 키워드로 실행합니다
- → 복잡한 문제일수록 깊은 사고 키워드를 사용하면 정확도가 향상됩니다

나만의 스킬 만들기

반복되는 업무 패턴을 스킬로 저장하여 재사용합니다

SKILL.md (예시: my-workflow) markdown

```
---
name: my-workflow
description: 코드 작성 후 리뷰와 커밋을 자동 수행
---

1. 요청받은 기능을 구현합니다
2. /code-review로 코드를 리뷰합니다
3. 리뷰 결과를 반영하여 수정합니다
4. 변경사항을 커밋합니다
```

→ .claude/skills/my-workflow/SKILL.md 경로에 저장합니다

→ 관련 요청이 오면 Claude Code가 자동으로 이 스킬을 활성화합니다

→ 단순 단축키(슬래시 커맨드)는 .claude/commands/my-workflow.md로 분리하여 /my-workflow로 호출합니다

플러그인 설치부터 사용까지

context7 플러그인을 설치하고, 라이브러리 문서를 자동으로 참조하는 실제 흐름입니다

/plugin marketplace add anthropics/claude-code

AI

공식 마켓플레이스를 추가했습니다.
개별 플러그인은 /plugin install <name>@claude-code 형식으로 설치합니다.

/plugin install context7@claude-code

AI

context7 플러그인을 설치했습니다. 라이브러리 최신 문서를 자동으로 참조해 최신 패턴을 제안합니다.

- 플러그인이 설치되면, 관련 작업 시 자동으로 활성화됩니다
- context7: 라이브러리 최신 문서를 참조하여 구버전 코드 방지

컨텍스트와 비용 관리

대화가 길어지면 품질이 떨어지고 비용이 증가합니다. 적절히 관리합니다

명령어	기능	사용 시점
/clear	대화 초기화	새로운 작업을 시작할 때, 실습 전환 시
/compact	대화 요약 후 압축	응답이 느려지거나 품질이 떨어질 때
/cost	토큰 사용량 확인	현재 세션의 비용을 확인할 때
Escape	응답 즉시 중단	잘못된 방향으로 진행될 때 (가장 중요)

- Claude가 잘못 가면: Escape로 중단하고, 구체적으로 방향을 다시 지시합니다
- 그래도 안 되면: /clear로 초기화하고 처음부터 명확하게 요청합니다
- 비용: Max 구독은 사용량이 제한되고, API 키는 토큰당 과금됩니다. /cost로 확인합니다

체크포인트

다음 섹션으로 넘어가기 전에 확인합니다

- ✔ 좋은 프롬프트의 3가지 요소를 말할 수 있습니까?
- ✔ Plan Mode와 Extended Thinking의 차이를 설명할 수 있습니까?
- ✔ 나만의 스킬을 만드는 방법을 알고 있습니까?

04

디자인 시스템과 프로젝트 설정

- ✓ 디자인 토큰의 개념과 활용법을 이해합니다
- ✓ CLAUDE.md와 /init으로 프로젝트를 설정합니다
- ✓ 커뮤니티 플러그인을 설치합니다

⏱ 25분

왜 디자인 시스템인가

일관된 디자인의 힘을 확인합니다

🎨

일관성

모든 화면에 같은 색상, 폰트, 간격을 적용해 전문적인 느낌을 줍니다.

⚡

생산성

디자인 결정을 반복하지 않습니다. 토큰을 참조하면 AI가 자동으로 적용합니다.

🔧

유지보수

CSS 변수 하나로 색상을 바꾸면 앱 전체에 반영됩니다.

디자인 토큰

색상, 타이포그래피, 간격을 체계적으로 정의합니다

design-tokens.css

```css
:root {
  /* Colors */
  --color-primary: #4F46E5;
  --color-secondary: #7C3AED;
  --color-success: #10B981;
  --color-danger: #EF4444;
  --color-bg: #F8FAFC;

  /* Typography */
  --font-family: 'Pretendard', sans-serif;
  --font-size-lg: 1.25rem;
  --font-size-base: 1rem;
  --font-size-sm: 0.875rem;

  /* Spacing */
  --space-xs: 0.25rem;
  --space-sm: 0.5rem;
  --space-md: 1rem;
  --space-lg: 1.5rem;
}
```

토큰 적용 예시

디자인 토큰이 실제 UI에 어떻게 적용되는지 확인합니다

요소	토큰	적용 결과
주요 버튼	--color-primary	보라색 배경의 버튼
위험 버튼	--color-danger	빨간색 배경의 삭제 버튼
카드 간격	--space-md	16px 패딩의 카드
제목 크기	--font-size-lg	20px 크기의 제목
본문 폰트	--font-family	Pretendard 서체 적용

design-guide-builder

디자인 가이드를 자동으로 생성하는 도구입니다

이 프로젝트에 맞는 모던하고 미니멀한 디자인 가이드를 만들어 주세요.

AI

디자인 가이드를 생성하겠습니다.

색상 팔레트, 타이포그래피 스케일, 컴포넌트 스타일을 정의한 design-guide.md 파일을 만들었습니다. 이 가이드를 CLAUDE.md에서 참조하면 모든 코드에 자동 적용됩니다.

공식 Skills 3종

공식 마켓플레이스가 설치되어 있어 바로 사용할 수 있습니다

🎨

/frontend-design

디자인 토큰을 분석하여 전문적인 UI를 생성합니다. 접근성과 반응형 디자인을 자동으로 고려합니다.

🔧

/feature-dev

영향 범위를 분석하고 단계별로 기능을 구현합니다. 설계, 구현, 테스트의 전체 흐름을 안내합니다.

🔍

/code-review

보안, 성능, 코드 품질을 자동 점검합니다. 발견된 이슈를 설명하고 수정을 제안합니다.

/frontend-design 실사용 데모

디자인 토큰을 적용하여 할일 앱 UI를 전문적으로 개선하는 실제 흐름입니다

/frontend-design 할일 목록 카드를 개선해 주세요. 우선순위별로 색상을 구분해 주세요.

AI

디자인 시스템을 분석하겠습니다.

design-guide.md의 토큰을 기반으로 카드를 재설계합니다.

1. 우선순위별 좌측 보더 색상 (높음: danger, 보통: primary, 낮음: muted)
2. 완료 항목 opacity 처리
3. 호버 시 그림자 효과
4. 모바일 스와이프 삭제 지원

구현을 진행하시겠습니까?

- 스킬은 기존 코드와 디자인 토큰을 분석한 뒤 작업합니다. 단순 프롬프트보다 정확합니다
- 실습 2에서 직접 사용합니다

CLAUDE.md와 /init

프로젝트의 규칙을 직접 쓰지 않아도 됩니다. Claude가 대화를 통해 자동 생성합니다

/init

AI

프로젝트를 분석하겠습니다.

이 프로젝트는 React + TypeScript 기반이며, Tailwind CSS를 사용합니다. 몇 가지 질문이 있습니다:

1. 테스트 프레임워크가 있습니까?
2. 코딩 컨벤션이 있습니까? (예: 함수형 컴포넌트만 사용)
3. 디자인 가이드 파일이 있습니까?

- 언제: 새 프로젝트를 시작하거나, 기존 프로젝트에 Claude Code를 처음 적용할 때 실행합니다
- 어떻게: /init을 입력하면 Claude가 코드를 분석하고 질문한 뒤 CLAUDE.md를 자동 생성합니다
- 왜: CLAUDE.md가 있으면 이후 모든 코드 생성에 프로젝트 규칙이 자동 적용됩니다

"호텔 안내문 1장 vs 200페이지" — CLAUDE.md를 짧게

AI는 한 장짜리 호텔 주의사항은 지키지만 200페이지 매뉴얼은 못 지킵니다 — 직원에게 일 시키듯 룰을 만들어 뿌립니다

📄

한 장짜리 호텔 안내

체크인 시간, 와이파이, 조식 위치 — 핵심만 담긴 한 장. 손님이 모두 읽고 지킵니다. CLAUDE.md도 이 정도가 가장 효과적입니다.

📚

200페이지 매뉴얼

모든 사례를 적었지만 누구도 끝까지 읽지 않습니다. AI도 같습니다 — 길수록 앞부분의 규칙을 잊거나 무시합니다. 규칙이 너무 많으면 분리·압축·삭제하십시오.

Rules 분리

.claude/rules/ 폴더로 규칙을 분리하여 관리합니다



CLAUDE.md

프로젝트 전체 규칙을 정의합니다. 기술 스택, 코딩 컨벤션, 디자인 가이드 참조를 포함합니다.



.claude/rules/

세부 규칙을 파일별로 분리합니다. frontend.md, api.md, testing.md 등으로 나누어 관리합니다.

공식 플러그인 마켓플레이스

Anthropic 공식 마켓플레이스를 통해 다양한 외부 도구와 연동할 수 있습니다

플러그인	제공	기능
vercel	Vercel	배포 연동, 빌드 상태 확인
supabase	Supabase	클라우드 DB, 인증, 스토리지 연동
figma	Figma	디자인 파일에서 코드 생성
context7	Upstash	라이브러리 최신 문서 자동 참조
playwright	Microsoft	브라우저 자동화, E2E 테스트

- 마켓플레이스 추가: /plugin marketplace add anthropics/claude-code
- 개별 플러그인 설치: /plugin install <name>@<marketplace>
- 라이브러리 문서 연동 플러그인은 오전 실습에서 함께 설치합니다

LO4 · 디자인 시스템

체크포인트

다음 섹션으로 넘어가기 전에 확인합니다

- ✔ 디자인 토큰(CSS 변수)의 역할을 설명할 수 있습니까?
- ✔ CLAUDE.md에 디자인 가이드를 참조하는 방법을 알고 있습니까?
- ✔ /init 명령어의 용도를 설명할 수 있습니까?

00

실습 2: Skills로 앱 업그레이드

- 요구사항과 디자인 시스템을 적용하여 앱을 재구성합니다
- /frontend-design 스킬로 전문적인 UI를 생성합니다
- 그냥 바이브와 Skills 적용의 차이를 체감합니다

⏱ 25분

비포/애프터

같은 앱이 Skills 적용으로 어떻게 달라지는지 확인합니다

😐

Before: 그냥 바이브

기본 HTML 스타일, 불명확한 기능, 일관성 없는 디자인, 사용성 미고려

🤩

After: Skills 적용

/frontend-design으로 디자인 토큰 기반 UI, Plan Mode로 계획적 구현, 요구사항 기반 명확한 기능

BLOCK 1 요구사항 생성 후 Plan Mode 적용

실습 절차

먼저 Claude에게 '할일 앱 요구사항을 정리해줘'라고 요청하여 requirements.md를 생성합니다. 그 다음 Shift+Tab을 두 번 눌러 Plan Mode로 전환하고, 요구사항에 따라 앱 재구성을 요청합니다.

```
할일 앱의 요구사항을 정리해줘. 대상은 직장인이고 빠른 입력과 우선순위 관리가 중요해.
```

관찰 포인트

Claude가 요구사항을 정리합니다. 이후 Plan Mode에서 재구성 계획을 검토하고 승인합니다.

2단계: 먼저 요구사항 생성, 그 다음 Plan Mode로 재구성

BLOCK 2 /frontend-design으로 UI 업그레이드

실습 절차

/frontend-design 스킬을 실행하여 전문적인 UI로 바꿉니다. 실행 전후를 비교합니다.

```
/frontend-design 디자인 가이드를 적용하여 할일 목록 UI를 전문적으로 개선해줘. 우선순위별 색상 구분 포함.
```

관찰 포인트

스킬이 기존 코드를 분석한 뒤 디자인 토큰을 적용해 일관된 UI를 만듭니다. 직접 지시하지 않아도 접근성과 반응형 디자인을 반영합니다.

스킬은 코드를 분석하고 맥락을 파악한 뒤 작업합니다

기술 원리 잠깐!

컴포넌트와 CSS 변수로 일관된 디자인을 만듭니다

- 💡 CSS 변수(Custom Properties): 하나의 값을 변경하면 전체 앱에 반영됩니다
- 💡 컴포넌트 분리: 버튼, 카드, 입력 필드를 재사용 가능한 단위로 만듭니다
- 💡 반응형 디자인: 미디어 쿼리로 화면 크기에 따라 레이아웃을 바꿉니다

실습 2 체크포인트

다음 섹션으로 넘어가기 전에 확인합니다

- ✔ Plan Mode로 계획을 검토한 뒤 구현을 진행하였습니까?
- ✔ /frontend-design으로 디자인 토큰 기반 UI가 완성되었습니까?
- ✔ 그냥 바이브(실습 1)와의 차이를 체감하였습니까?

06

plugin 활용 — DB와 배포의 두 장벽 해소

- ✓ 명령 한 줄로 Supabase plugin을 설치하고 자연어로 DB를 추가합니다
- ✓ 명령 한 줄로 Vercel plugin을 설치하고 자연어로 실제 URL을 만듭니다
- ✓ 환경변수와 시크릿을 안전하게 분리하는 원칙을 익힙니다

⏱ 30분

비개발자가 직면하는 두 가지 진입장벽

기획과 UI는 자연어만으로 구현할 수 있지만, DB 연동과 배포 단계에서 자주 막힙니다

벽	전형적인 막힘	plugin 설치 후
DB 연동	Supabase 가입, 키 발급, SQL 학습을 모두 직접 진행합니다	한 줄 프롬프트로 DB를 연결하고 마이그레이션을 자동 처리합니다
배포	Vercel CLI 설치, 토큰 발급, 환경변수 등록을 직접 진행합니다	한 줄 프롬프트로 GitHub를 연결하고 배포와 env 등록을 자동 처리합니다

데이터베이스(DB)의 역할과 필요성

DB는 화면(메모리)에 잠시 떠 있는 정보를 영구 저장하는 공간입니다. 새로고침하거나 다시 접속해도 데이터가 보존됩니다

상황	DB 없음 (브라우저 메모리만 사용)	DB 있음 (Supabase 등 외부 저장소)
페이지 새로고침	입력한 데이터가 모두 사라집니다	데이터가 그대로 유지됩니다
다른 기기에서 접속	이전에 입력한 데이터를 확인할 수 없습니다	어떤 기기에서 접속해도 동일한 데이터를 확인할 수 있습니다
여러 사용자가 사용	사용자마다 데이터가 분리되어 서로 공유할 수 없습니다	모든 사용자가 동일한 데이터를 공유합니다
비유로 이해	메모를 기억에만 남겨 둔 상태입니다	메모를 노트에 기록하여 보관한 상태입니다

배포와 로컬 html 파일의 차이

배포는 본인 PC에 있던 산출물을 공개 URL에 게시해 누구나 어디서나 접속하게 하는 작업입니다

항목	로컬 html 파일 (file://)	Vercel 배포 (https://)
주소 형태	file:///Users/.../index.html (본인 PC 내부 경로)	https://my-app.vercel.app (공개 도메인)
다른 사용자 접속	본인 PC에서만 사용할 수 있습니다	URL을 전달하면 어디서나 접속할 수 있습니다
모바일 접속	원칙적으로 지원되지 않습니다	스마트폰 브라우저에서 동일하게 동작합니다
코드 수정 반영	수정한 PC에만 반영됩니다	git push 한 번으로 모든 사용자에게 자동 반영됩니다
외부 DB 연결	브라우저 보안 정책으로 인해 외부 DB 연결이 제한됩니다	환경변수에 키를 등록하면 안정적으로 연결됩니다

DB와 배포는 서로 다른 차원의 과제입니다

'DB를 연결하지 않으면 배포의 의미가 없는 것 아닙니까'라는 질문이 있었습니다. 데이터 보존(DB)과 접근성 확보(배포)는 별개이므로 각각 해결해야 합니다

배포 여부 (행) / DB 여부 (열)	DB 없음	DB 있음
배포하지 않음 (file://)	본인 PC에서만 동작합니다. 새로고침 시 데이터가 사라지므로 가장 초기 상태에 해당합니다	본인 PC에서만 동작합니다. 데이터는 보존되나 외부 공유는 불가능합니다
배포 완료 (vercel.app)	URL은 공유 가능하나 사용자마다 데이터가 분리됩니다. 새로고침 시 사라집니다 (방명록 등 일시적 용도)	URL 공유만으로 누구나 접속이 가능하며 모든 사용자가 동일한 데이터를 확인합니다. 실서비스 형태입니다
한 줄 정리	'공유'와 '보존'은 서로 다른 차원의 요구사항입니다	DB는 보존, 배포는 공유를 담당합니다. 두 가지를 모두 갖추어야 완성된 서비스가 됩니다

직접 실행하고 확인하는 절차의 필요성

AI가 '완료했습니다'라고 응답해도 실제로는 정상 동작하지 않을 수 있습니다. 사람이 직접 최종 검증해야 합니다

검증 시점	AI 응답 예시	실제 실행 결과 예시
기능 추가 요청 직후	"요청하신 기능을 완료했습니다. 정상 동작합니다"	버튼을 클릭하면 오류 메시지가 나타날 수 있습니다
DB 연동 직후	"Supabase 연결을 확인했습니다"	환경변수가 빠져 저장되지 않을 수 있습니다
배포 직후	"https://*.vercel.app에 정상 배포했습니다"	PC에서는 동작하지만 모바일에서는 레이아웃이 깨질 수 있습니다
비개발자도 할 수 있는 검증	버튼 클릭, 데이터 입력, 새로고침, 모바일에서 접속	오류 메시지를 그대로 복사해 Claude에 전달하면 해결됩니다

명령을 입력하기 전, 매 단계의 목적을 확인합니다

설치와 입력만 따라 하면 학습 효과가 제한적입니다. 각 단계마다 '이 절차를 수행하는 이유'를 한 줄로 정리합니다

진행 단계	수행 목적	확보되는 결과
Supabase plugin 설치	DB 연동 작업을 자연어 한 줄로 맡깁니다 (가입과 SQL 학습 단계가 생략됩니다)	설치 1회로 영구 활용 가능
Supabase 키 등록	앱이 데이터를 저장할 DB의 주소와 인증 정보입니다	DB 연결 활성화
Vercel plugin 설치	GitHub 업로드, 배포, 환경변수 등록을 자연어 명령으로 맡깁니다	설치 1회로 영구 활용 가능
Vercel 배포	공개 URL을 만들어 외부 사용자와 공유합니다 (본인 PC 밖에서도 접속할 수 있습니다)	라이브 URL 확보
배포 후 직접 검증	실제 동작을 사람이 확인해 AI 응답에만 의존하지 않습니다	오류 발생 시 즉시 해결

plugin 마켓플레이스 — Claude Code의 앱스토어

사전 제작된 워크플로우(슬래시 커맨드, Skill, MCP, 서브에이전트)를 한 번의 명령으로 설치합니다

claude (REPL 안에서 입력) bash

```
# 1) 공식 마켓플레이스 등록 (1회)
/plugin marketplace add anthropics/claude-plugins-official

# 2) plugin 설치 (1회)
/plugin install supabase
/plugin install vercel

# 3) 설치 확인
/plugin list
```

MCP와 Skill — 컨텍스트 점유 방식이 다릅니다

MCP는 세션 시작부터 컨텍스트를 점유합니다. Skill은 필요할 때만 적재됩니다

항목	MCP	Skill
컨텍스트 점유	세션 시작과 동시에 모든 도구 정의를 적재합니다 (컨텍스트 사용량 큼)	필요할 때만 lazy load 방식으로 적재합니다 (평시 토큰 사용량 0)
역할	외부 시스템 연결을 담당합니다 (DB, GitHub, Slack 등 실제 작업 수행)	재사용 가능한 워크플로우를 정의합니다 (코드 리뷰, 디자인 가이드 등)
예시	supabase (DB 작업), vercel (배포), github (이슈·PR 관리)	code-review, frontend-design, skill-creator
비개발자 권장 사항	Plugin이 자동 등록·관리하므로 직접 관리할 필요가 없습니다	반복 작업을 찾으면 직접 제작할 수 있습니다

Supabase plugin — 자연어 한 줄로 DB 연동

1회 설치 이후, 대화창에서 자연어 한 줄로 DB, 인증, 파일 저장 기능을 제어할 수 있습니다

의도	프롬프트 한 줄
영속성 추가	할일을 Supabase에 저장하도록 연동해줘
새 필드 추가	할일에 마감일(due_date) 필드를 추가해줘
검색 기능	제목으로 검색하는 기능을 추가해줘
사용자 분리	내가 만든 할일만 보이도록 사용자 인증을 붙여줘

Vercel plugin — 자연어 한 줄로 배포

GitHub 저장소 생성, 연결, 배포, 환경변수 등록 절차를 자연어 한 줄로 일괄 위임합니다

의도	프롬프트 한 줄
첫 배포	이 프로젝트를 GitHub에 올리고 Vercel에 배포해줘
환경변수 등록	Supabase 키를 Vercel 환경변수에 등록하고 재배포해줘
URL 확인	배포된 URL 알려줘
재배포	지금 변경사항을 다시 배포해줘

전체 흐름 한 장 요약 — 1회 설치, 매번 자연어 한 줄

설치는 한 번이면 충분하며, 이후에는 자연어 한 줄로 사용할 수 있습니다. 비개발자는 두 영역만 알면 됩니다

bash · 작업 흐름 요약 bash

```bash
# 1회 초기 설정 (강의 초반에 1회 수행)
/plugin marketplace add anthropics/claude-plugins-official
/plugin install supabase
/plugin install vercel

# 매 프로젝트 (자연어 한 줄)
> 할일을 Supabase에 저장되게 해줘          # DB 연결 완료
> 이 프로젝트를 Vercel에 배포해줘           # 라이브 URL 확보
> 환경변수 등록하고 재배포해줘              # 시크릿 안전 적용
```

환경변수와 시크릿 — 코드와 분리하여 관리합니다

DB 비밀번호와 API 키는 .env 파일로 분리하고, 배포 환경에는 plugin이 자동 등록합니다

항목	관리 위치
로컬 개발 환경	.env.local 파일 (개인 노트북에만 존재)
Git 저장소	업로드하면 안 됩니다 (.gitignore에 자동 등록)
배포 환경	Vercel 환경변수 (plugin이 자동 등록·재배포 수행)
핵심 원칙	키와 비밀번호는 코드와 분리하여 관리합니다

plugin 사용 전후의 작업 비교

plugin은 비개발자의 진입장벽을 없애는 도구입니다

과정	plugin 미사용	plugin 설치 후
DB 연동	Supabase 검색, 키 발급, SQL 학습에 30분 이상 소요됩니다	자연어 한 줄로 즉시 적용 가능합니다
배포	Vercel 가입, CLI 설치, 토큰 발급을 직접 수행합니다	자연어 한 줄로 GitHub를 연동하고 자동 배포합니다
환경변수	등록 위치와 방법을 별도로 학습해야 합니다	plugin이 자연어로 절차를 안내합니다
체감 소요 시간	검색과 디버깅에 1~2시간이 소요됩니다	핵심 기능 구현에 집중할 수 있습니다

체크포인트

다음 섹션으로 넘어가기 전에 확인합니다

- ✔ /plugin marketplace add → /plugin install 흐름을 직접 실행할 수 있습니까?
- ✔ Supabase plugin이 설치된 상태에서 'DB에 저장되게 해줘' 한 줄로 영속성을 붙일 수 있습니까?
- ✔ Vercel plugin이 설치된 상태에서 '배포해줘' 한 줄로 실제 URL을 얻은 적이 있습니까?
- ✔ 키와 비밀번호를 코드 옆에 두지 않는 이유를 한 문장으로 설명할 수 있습니까?

07

실전 통합 — plugin으로 완성

- ✓ Supabase plugin을 사용하여 자연어 한 줄로 데이터를 계속 저장합니다
- ✓ Vercel plugin을 사용하여 자연어 한 줄로 라이브 URL을 확보합니다
- ✓ 오류를 해결하고 동료에게 30초간 시연합니다

⏱ 50분

BLOCK 1 Supabase plugin — DB 연동 (자연어 한 줄)

실습 절차

오전에 설치한 Supabase plugin을 그대로 활용합니다. 본인 Supabase 프로젝트의 개발용 키를 적용하여 한 줄 프롬프트를 실행합니다.

```
이 앱의 데이터를 Supabase에 저장하도록 연동해줘. 새로고침해도 데이터가 유지되게.
```

관찰 포인트

plugin이 마이그레이션 SQL 작성과 .env.local 등록까지 자동으로 처리합니다. 새로고침 후에도 데이터가 유지되는지 확인하십시오.

수행 이유 — 새로고침하거나 다시 접속해도 데이터를 보존하기 위해 외부 저장소(DB)에 맡깁니다. 비개발자가 마주하는 첫 번째 진입장벽을 해소합니다.

BLOCK 2 Vercel plugin — 배포 (자연어 한 줄)

실습 절차

오전에 설치한 Vercel plugin을 그대로 활용합니다. GitHub 저장소 생성, 연결, 배포, 환경변수 등록까지 한 줄 프롬프트로 위임합니다.

```
이 프로젝트를 GitHub에 올리고 Vercel에 배포해줘. Supabase 키도 환경변수로 등록해줘.
```

관찰 포인트

plugin이 GitHub 저장소를 생성하고 Vercel과 연결합니다. 빌드가 끝나면 https://*.vercel.app 형태의 URL을 발급하며, 모바일에서도 접속할 수 있습니다.

수행 이유 — 외부 사용자도 접속할 수 있는 공개 URL을 만듭니다. 비개발자의 두 번째 진입장벽을 해소합니다.

BLOCK 3 디버깅 및 UI 정돈

실습 절차

시연 전 마무리 단계입니다. 오류 메시지를 그대로 Claude에 전달해 해결하고, 더미 데이터, 여백, 색상을 정돈해 완성도를 높입니다.

```
배포된 페이지에서 [에러 메시지 또는 어색한 UI]가 보여. 고쳐줘.
```

관찰 포인트

오류를 해결하고 시연할 수 있도록 UI를 정돈합니다. 시연용 더미 데이터 3건을 미리 입력하면 첫인상이 좋아집니다.

수행 이유 — 실제 화면을 사람이 확인해야 찾을 수 있는 오류가 있습니다. 오류 메시지를 그대로 전달하는 것이 가장 빠른 해결 방법입니다.

동료 시연 — 30초 단위

조별로 1~2명이 본인 라이브 URL을 열고 30초간 시연합니다. 동료의 박수로 마무리합니다

- 본인 Vercel URL을 화면에 띄우거나 옆자리에서 보여 줍니다
- 한 줄 소개 — 'OOO이 OOO를 수행할 때 OOO하게 지원하는 앱입니다'
- 핵심 기능 1~2개를 30초 이내에 클릭 시연으로 보여 줍니다
- 동료의 박수와 한 마디 코멘트로 마무리합니다

체크포인트

다음 섹션으로 넘어가기 전에 확인합니다

- ✔ Supabase plugin을 자연어 한 줄로 적용하여 데이터 영속성을 확보하셨습니까?
- ✔ Vercel plugin을 자연어 한 줄로 적용하여 본인 라이브 URL을 확보하셨습니까?
- ✔ 모바일에서도 본인 URL 접속이 가능합니까?
- ✔ 동료에게 30초 분량의 시연을 수행하셨습니까?

00

종합 정리

- ✓ 오늘의 학습을 정리합니다
- ✓ 동료와 리뷰를 교환합니다
- ✓ 다음 학습 경로를 안내합니다

⏱ 50분

앱 성숙도 점검 (학습용 기준)

오늘 만든 앱이 어느 단계에 도달했는지 확인합니다. 실제 운영·CI/CD·SRE는 별도 학습이 필요합니다.

단계	설명	체크
1. MVP	기본 기능이 동작합니다	✓
2. 디자인	디자인 시스템이 적용되었습니다	✓
3. 데이터	Supabase로 데이터가 저장되고 유지됩니다	✓
4. 배포	실제 URL로 접근 가능합니다	✓
5. 운영	모니터링, 로깅, CI/CD가 구성되었습니다	

자동화 성숙도 사다리

본 과정에서 도달한 단계와 다음에 진행할 단계를 정리합니다

단계	내용	본 과정 도달 수준
1. 패턴화	반복 작업의 흐름을 인식하고 동일 프롬프트로 재현합니다	첫 바이빙과 기획서 작성 패턴 체득
2. 룰 (Rule)	CLAUDE.md에 프로젝트 규칙을 명시하여 AI에게 일관되게 작업을 위임합니다	본인 프로젝트의 CLAUDE.md 작성 완료
3. 스킬 (Skill)	반복 워크플로우를 SKILL.md로 추출하여 자연어 한 줄로 호출합니다	Supabase·Vercel plugin 호출로 간접 체험 (다음 단계)
4. Plugin	본인 자산을 묶어 마켓플레이스를 통해 사내에 공유합니다	후속 학습 — 동일 작업이 3회 반복되는 시점부터 추출 시작

AX 도입 ROI 사다리 — 시작점

모든 업무가 아니라 자동화가 필요한 영역만 AX로 전환하고, 나머지 시스템은 단순화해야 ROI가 가장 높습니다

단계	전략	ROI
0. 분석 생략	레거시와 업무를 분석하지 않고 AI만 도입한 경우 — 겉모습만 바뀌어 효과가 작습니다	ROI 음수
1. 표면 자동화	회의록 정리, 문서 요약 등 단발성 작업을 AI에게 위임합니다 (초기 도입에 적합)	ROI 낮음
2. 핵심 자동화	반복 업무 중 자동화가 실제로 필요한 영역만 선별하여 AX로 전환합니다	ROI 가장 높음
3. 시스템 최소화	자동화가 어려운 영역은 시스템을 단순화하여 사람이 직접 신속하게 처리합니다	ROI 높음

회고

과정에서 배운 내용을 정리합니다

Q. 오늘 가장 인상적이었던 순간은 무엇입니까?

Q. 내일 실무에서 가장 먼저 시도할 것은 무엇입니까?

Q. 바이브코딩이 아직 궁금합니까?

비개발자의 강점

"왜 이것을 만들었는가"를
가장 잘 설명할 수 있는 사람

구현은 AI가 빠르게 합니다. 비개발자가 만드는 차이는
문제 정의·사용자 이해·설득력에서 나옵니다.

본 과정 핵심 정리

바이브코딩 전 과정에서 배운 내용을 정리합니다

- 바이브코딩: 자연어로 소프트웨어를 만드는 새로운 개발 방식입니다
- Claude Code: 터미널 기반 AI 코딩 도구를 설정하고 사용했습니다
- 요구사항과 UX: 명확한 요구사항으로 결과 품질을 높입니다
- 디자인 시스템: 디자인 토큰과 CLAUDE.md로 전문적인 UI를 구현합니다
- Plan Mode와 Skills: 계획을 검토한 뒤 반복적으로 개선합니다
- Supabase plugin: 한 줄 프롬프트로 데이터베이스를 연동합니다
- Vercel plugin: 한 줄 프롬프트로 실제 공유 가능한 URL을 생성합니다

실무 도입 가이드

향후 1~2주 동안 본인 업무에 시범 적용할 실천 항목입니다

💡

본인 업무 아이디어 정리

이번 주 중 본인 업무에서 바이브코딩으로 검증할 아이디어 1건을 정리합니다.

🚀

주간 프로토타입 제작

주 1회 프로토타입을 제작하여 Vercel URL로 공유합니다. 기획 역량이 뚜렷하게 향상됩니다.

🤝

개발 조직과의 소통

기획 의도를 프로토타입으로 전달하여 개발 조직과 더 정확하게 소통합니다.